P2 — Building a Lossless Compression Codec

1. Design and implementation

We implemented a **lossless codec** combining canonical Huffman coding with optional delta prediction. It accepts raw bytes and NumPy arrays, and preserves the exact values, shape, and data type during reconstruction. Its stages are:

1. **Input handling:** Byte sequences are compressed directly. For arrays, the codec stores the original shape and data type so they can be reconstructed.
2. **Prediction:** For RGB `uint8` arrays of shape `(H, W, 3)`, the codec can use no prediction, horizontal differences (from the previous pixel in a row), vertical differences (from the pixel above), or spiral differences (from the previous pixel in a spiral traversal). Differences are taken modulo 256, making every transform reversible. Prediction can make neighboring values more repetitive and easier to code. With `auto`, the codec tries all four modes and chooses the smallest complete stream. Other array formats use no predictor.
3. **Huffman encoding:** `Counter` counts byte frequencies. A priority queue (`heapq`) repeatedly merges the two least frequent nodes to construct a Huffman tree. Frequent bytes receive shorter codes. Canonical Huffman codes are then generated from the code lengths, allowing the decoder to rebuild the codes without storing the entire tree.
4. **Stream construction:** The encoded bits are padded to full bytes. The array stream stores a format marker, metadata length, JSON metadata (shape, dtype, predictor), Huffman header (original byte count and symbol count), symbol/code-length table, and coded payload. These fields are required for independent decoding.
5. **Metadata:** We store the array shape, data type, and selected predictor as compact JSON using minimal separators. This reduces unnecessary metadata overhead while preserving the information needed for exact reconstruction.
6. **Decoding:** The decoder reconstructs canonical codes, decodes the bytes, restores the array shape and dtype, then reverses any delta prediction by cumulative addition modulo 256. No quantization or information loss occurs.

2. Correctness tests

The `check_roundtrip()` function compresses and decompresses each input. Byte sequences must match exactly; NumPy arrays must also retain their original type, shape, and dtype, and pass `np.testing.assert_array_equal`. Tests covered empty bytes and arrays, single bytes/pixels, tiny images, repeated values, all 256 byte symbols, random and text-like bytes, gradients, grayscale arrays, `int16` and `float32` arrays. All reported round-trip tests passed. Each predictor mode (`none`, horizontal, vertical, spiral, `auto`) also passed.

Input	Original	Coded
Empty bytes	0 B	13 B
Repeated symbol	1,000 B	140 B
Horizontal gradient	24,576 B	3,164 B
Random RGB image	12,288 B	12,875 B
`float32` array	80 B	244 B

Two additional unseen files were tested. For `unseen.png`, `auto` reduced the RGB array from 583,200 to 146,410 bytes, selecting vertical prediction. For noise-like `unseen2.png`, it reduced 88,800 to 87,001 bytes, selecting no prediction. Both files passed exact reconstruction tests in all predictor modes. These results also show that the best predictor depends on image content.

3. Genuine byte count and baseline

Compressed size is measured using `len(compressed)`, including **all** headers, metadata, symbol tables, padding, and payload. We report compression ratio as $\frac{\text{original bytes}}{\text{compressed bytes}}$ and bits per pixel as $8 \cdot \frac{\text{compressed bytes}}{\text{number of pixels}}$.

For `kirmizi (1).jpg`, the raw decoded RGB array was 1,080,000 bytes; our output was **230,647 bytes** (4.68:1; 5.13 bits/pixel). This comprised **599 bytes of side information** (4-byte marker, 4-byte metadata-length field, 66-byte JSON metadata, 13-byte Huffman header, and 512-byte symbol table) plus **230,048 payload bytes**. The code asserts that the parts sum to the complete stream size.

PNG is our **lossless baseline**: both methods compress the same decoded RGB arrays and preserve them exactly. JPEG at quality 95 was also measured, but it is lossy and is not an equivalent correctness comparison.

Pistachio image	Our codec	PNG
(1)	230,647 B	94,867 B
(10)	221,718 B	97,616 B
(11)	241,131 B	114,154 B
(12)	212,245 B	81,907 B
(13)	234,973 B	105,618 B

All five RGB images had raw sizes of 1,080,000 bytes. Our method reduced each to around 20–22% of the raw size, but PNG was smaller for every tested image. The custom codec demonstrates successful prediction and entropy coding, not state-of-the-art compression.

4. Failure case and limitations

Compression can **increase** size when data has little redundancy. Random bytes increased from 100,000 to 100,525 bytes, and a random RGB image increased from 12,288 to 12,875 bytes. Huffman codes provide little saving when symbol frequencies are nearly uniform, while headers and the symbol table still take space. The method is therefore unsuitable when small or high-entropy inputs must always become smaller. Delta prediction is also limited to RGB `uint8` arrays; on the noise-like unseen image, every delta predictor was worse than no prediction.

5. AI declaration

NTNU

Declaration of AI aids and tools

Have any AI-based aids or tools been used in the creation of this report?

✓ No Yes

If yes: please specify the aid/tool and area of use below.

Text

Spell checking. Are parts of the text checked by: Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinka AI or similar tools?

✓ **Text-generation.** Are parts of the text generated by: ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr or similar tools?

Writing assistance. Are one or more of the report's ideas or approach suggested by: ChatGPT, Google Bard, Bing chat, YouChat or similar tools?

If no, sign document. If yes, use of text aids/tools apply to this report -please specify usage here:

The document has only been language washed to help formulate things easier

---

Codes and algorithms

✓ **Programming assistance.** Are parts of the codes/algorithms that i) appear directly in the report or ii)have been used to produce results such as figures, tables or numerical values been generated by: GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPTEngineer, ChatGPT, Google Bard or other tools.

If yes, use of programming assistance aid/tools apply to this report - please specify usage here:

AI has been used as a guider rather than a generator, helping us as we write the code

---

Images and figures

Image generation. Are one or more of the reports images/figures generated by: Midjourney, Jasper, WriteSonic, Stability AI, Dall-E or similar tools?

If yes, use of image generator aids/tools apply to this report – please specify usage here:

Other AI aids or tools. Have you used other types of AI aids or tools in the creation of this report?
If yes, please specify usage here:

✓ I am familiar with NTNU's regulations on artificial intelligence. I declare that any use of AI aids or tools are explicitly stated i) directly in the report or ii) in this declaration form.

09-11-2026/NTNU In Gjøvik

Signature/Date/Place

Ole Marius Glomsrud
Vinjar Aurstad Berland